Log2Root: Log-Guided Root-Cause Code Localization Beyond the Logging Statement

Anonymous Author(s)

***Abstract*—When a production issue is first reported, engineers typically hold only a natural-language report, a noisy log bundle and the source code; failing tests, coverage and debugger state are not yet available. Existing fault localization relies on such execution data, and log-based root cause analysis in AIOps does not rank defective source-code regions. Worse, the logging statement that reported a failure is often not where the fault lies. We address the ICSE 2027 Industry Challenge on log-guided root-cause code localization with three contributions. First, LogRootBench is, to our knowledge, the first public benchmark for execution-free, log-guided localization with method-level ground truth and module labels: 1,002 issues from 11 Apache Java systems, derived from fix commits, with leakage masking and labelled splits for symptom≠cause, non-crash, recent (2025–2026) and noisy-log conditions. Second, Log2Root is a training-free pipeline that selects failure-relevant log evidence, retrieves candidate code bottom-up with a hybrid log-to-code retriever (logging-statement templates, BM25 and dense code embeddings), adds candidates from a top-down, issue-guided localizer, and asks an LLM to output the affected module, a ranked list of suspicious methods and an explanation whose cited log lines and methods are checked to exist. Third, an empirical study against nine baselines. Without any LLM call, Log2Root's static stage beats all non-LLM baselines at method level (Acc@5 0.624 vs. 0.537). With `gpt-5-mini`, Log2Root reaches 0.515 method Acc@1, 0.720 Acc@5 and 0.606 MRR, significantly above the strongest baseline at Acc@5 (0.663) and marginally at MRR (0.564), but not at Acc@1 (0.491); at module level that baseline is better. The lead comes from combining bottom-up and top-down candidates: without top-down candidates, Log2Root is only level with that baseline across four backbones, and on the open-weight `gpt-oss-120b` the full method improves over that variant but does not beat the baseline. When the symptom site is not the cause, ranking logging sites almost never works (0.012 Acc@1), while LLM reasoning recovers part of the gap (Log2Root 0.354, the strongest baseline 0.335). The benchmark, code and cached model outputs are released so that every number can be recomputed offline.**

***Index Terms*—Log analysis, fault localization, root cause analysis, retrieval-augmented generation, benchmark.**

I. Introduction

Logs are often the first, and sometimes the only, evidence engineers have when a production failure is reported. A typical ticket contains a natural-language description of unexpected behaviour plus a pasted or attached log bundle, while the source code is available in version control. What is missing is execution information: when a crash report is filed, a fault-triggering test is rarely available. Rafi et al. found one for only 3.33% of the crash-report bugs they studied in Defects4J [1], and debugger state or traces from the failing run are seldom recoverable. Engineers therefore read the logs, guess which events are causal, find the code that emitted them and work backwards to the defect. This is slow and error-prone in large multi-module systems.

Automating this workflow is the goal of the ICSE 2027 Industry Challenge "Log-Guided Root-Cause Code Localization from Production Issues". Given an issue report, logs and a source snapshot, a solution must output (i) the affected module, (ii) a ranked list of suspicious code regions and (iii) an explanation linking symptoms, selected log events and code. It must rely on a retrieval model or retrieval-augmented method that jointly understands log and code semantics, and it must localize the root cause rather than the logging statement that reported the symptom. Existing techniques cover only parts of this. Source-level fault localization, including recent LLM-based approaches such as LLM4FL [2], AgentFL [3], AutoFL [4] and FaR-Loc [5], takes failing tests or coverage as input. Log-based root cause analysis in cloud AIOps, such as RCAgent [6] and RCACopilot [7], produces root-cause categories, services or summaries rather than ranked code regions.

Why the problem is hard. The challenge names three difficulties, and each of them breaks a naive solution. *(1) Evidence-to-code retrieval.* A log line is a rendered string whose constant part comes from a logging statement and whose meaning is implicit in the surrounding code [8]; modules share similar templates, so lexical matching retrieves many plausible but wrong candidates. *(2) Noisy and incomplete evidence.* Bundles mix retries, background activity and downstream errors produced after the original fault [9], while the decisive event may be absent or appear only as a generic timeout. *(3) Symptom versus root cause.* A log shows where a failure was *detected*, not where it *originated*. Ranking the methods that emitted the error lines fails whenever the defect lies upstream, in code that wrote a corrupted value, or in a component whose misbehaviour surfaces elsewhere; many such issues do not crash at all [10].

The gap. Two lines of work come closest. Pathidea [11] reconstructs execution paths from the logs in bug reports over a static call graph to improve IR-based bug localization. It ranks *files*, matches logs lexically, gives no explanation and releases only its data [12]. COCA [13] combines issue reports, logs and code: it backtracks logging sources, reconstructs execution paths and prompts an LLM. Its output, however, is a root-cause summary plus *responsible components*, which include

non-code concepts, rather than ranked code regions. Its 106-issue dataset and code are not public. LLM issue localizers such as Agentless [14], SweRank [15] and LocAgent [16] rank functions from issue text but treat logs as ordinary prose. As a result, no public benchmark combines issue text, logs, a pre-fix snapshot, *method-level* ground truth and labels that separate symptom sites from root causes. In addition, no prior work evaluates log-to-code retrieval as a stage of its own, and none reports Recall@K, MRRand Acc@K at method level for execution-free, log-guided localization on public data.

This paper. We close the gap with a benchmark and a method (Fig. 1). LogRootBench upgrades the public Pathidea data [12] to method-level ground truth with module labels by analysing the pre-fix snapshot and fix diff of each issue. It also adds freshly mined post-2025 issues, masks root-cause leakage in issue text, and labels symptom≠cause, non-crash and noisy-log conditions. Log2Root is a training-free pipeline that uses no execution data. It (a) selects failure-relevant log evidence; (b) retrieves code bottom-up with a hybrid retriever that matches log lines to logging-statement templates and fuses this with lexical and dense code retrieval; (c) expands from the logging sites to root-cause candidates along static call and data-flow relations; (d) adds candidates from a top-down, issue-guided localizer that navigates the repository structure, because many root causes are far from every logged method; and (e) prompts an LLM, in a Tracer→Debugger→Reviewer decomposition, to rank candidates while explicitly separating the symptom site from the cause. Every cited log line and method in its explanation is checked against the input.

Contributions.

- **C1** **LogRootBench**, to our knowledge the first public benchmark for execution-free, log-guided root-cause localization with method-level ground truth, module labels and symptom≠cause, non-crash, recent (2025–2026) and noisy-log splits (1,002 issues, 11 systems; Sec. III).
- **C2** **Log2Root**, a pipeline that combines bottom-up log-to-code retrieval with top-down candidate expansion and grounded LLM reasoning to produce all three required outputs; its static stage alone is a strong, cheap baseline (Sec. IV).
- **C3** **An empirical study** against nine execution-free IR, neural and LLM baselines, with cross-checks on further LLM backbones, covering retrieval quality, symptom-versus-cause behaviour, an error analysis, log perturbations, explanations and memorization (Secs. V–VI).
- **C4** **A re-runnable, anonymized artifact**: benchmark regeneration scripts, derived labels, code and cached model outputs, from which every reported number can be recomputed offline[1].

Main findings. (i) Without any LLM call, Log2Root's static stage beats every non-LLM baseline at method Acc@5 and MRR on all 952 test issues (Acc@5 0.624 vs. 0.537). (ii) With `gpt-5-mini`, Log2Root is the most accurate system at method level (0.515 Acc@1, 0.720 Acc@5, 0.606 MRR) and significantly better than the strongest baseline, an Agentless-style localizer, at Acc@5 (marginally at MRR); its Acc@1 gain is not significant, and at module level the baseline is better. (iii) Bottom-up log-anchored retrieval and top-down navigation fail on different issues: without top-down candidates, retrieval causes 77% of Log2Root's top-5 misses and Log2Root is only level with the baseline across four backbones; combining both sources produces the lead on `gpt-5-mini`, while on `gpt-oss-120b` it improves Log2Root but does not beat the baseline. (iv) When the symptom site is not the cause, ranking logging sites almost never finds the fault (0.012 Acc@1), while LLM reasoning recovers part of the gap (Log2Root 0.354, Agentless-style 0.335).

Section II formalizes the task and maps each challenge requirement to the place where we address it (Table I).

[1]Included as anonymized supplementary material and archived permanently with a public DOI upon acceptance.

II. Problem Formulation

Input. An instance is a triple $x = (I, L, S)$. I is the issue report, consisting of its title and description. Text written after the fix and passages that reveal the root cause are masked (Sec. III-C). $L = \langle \ell_1, \ldots, \ell_n \rangle$ is the log evidence attached to the report: runtime log lines and stack-trace frames, each with a stable identifier (`L1`, `L2`, ...) and a kind (*log*, *stack* or *other*). S is the source snapshot at the parent of the fixing commit. No execution information is available: there are no tests, no coverage, no traces from the failing run and no debugger state. A solution may analyse S statically.

Code units. Let $\mathcal{M}(S)$ be the set of methods and constructors in the non-test Java files of S. Each method m has a file $f(m)$, an enclosing class and a line span. Its module $\mathrm{mod}(m)$ is the directory of the nearest ancestor `pom.xml` of $f(m)$, i.e., the Maven module (e.g., `hadoop-hdfs-project/hadoop-hdfs`), or the top-level directory if there is none. The ground truth $G \subseteq \mathcal{M}(S)$ is the set of non-test methods modified by the fix. The ground-truth files and modules are $f(G)$ and $\mathrm{mod}(G)$.

Output. A solution returns (i) a ranked list of modules, whose top entry is the predicted *affected module*; (ii) a ranked list $R = \langle r_1, \ldots, r_k \rangle$ of suspicious methods with line spans, from which a file ranking is derived by order-preserving deduplication; and (iii) an explanation $E = (\text{text}, L_E, M_E)$, where $L_E \subseteq L$ are the cited log lines and $M_E \subseteq \mathcal{M}(S)$ the cited methods.

Logging sites, symptoms and root causes. For a log line ℓ, its *logging site* $\mathrm{anchor}(\ell) \in \mathcal{M}(S)$ is the method containing the logging statement that emitted it, determined by matching ℓ against static templates (Sec. III-D). For a stack trace, the logging site is the method of the top application frame. An instance with at least one resolved logging site is *symptom≠cause* if no ground-truth method is a logging site of any evidence line in L. In that case, ranking the logging sites first cannot be correct, and the root cause must be inferred from the code.

TABLE I
REQUIREMENTS TRACEABILITY: EACH CHALLENGE REQUIREMENT (R1–R9) AND WHERE THIS PAPER SATISFIES AND EVALUATES IT.

#	Challenge requirement	How we satisfy it	Where
R1	Input: issue report, logs, source snapshot; no execution information	LogRootBench instances (I, L, S); static analysis only	§II, §III
R2	Output (i): affected module	Explicit, validated module prediction; module-level metrics	§IV-D, Tab. IV
R3	Output (ii): ranked suspicious code regions	Ranked methods with line spans; file and method metrics	§IV-D, Tab. IV
R4	Output (iii): explanation linking symptoms, log events, code	Explanation citing log IDs and methods; grounding rate; case study	§IV-D, §VI-E
R5	Log-to-code retrieval or RAG over log and code semantics	Hybrid template/BM25/dense retriever, evaluated as its own stage	§IV-B, §VI-B
R6	Localize root cause, not the logging statement	Propagation, top-down expansion, Debugger; symptom≠cause split; logging-statement-only baseline	§IV-C, §VI-C
R7	Noisy/incomplete logs; non-crash issues	Evidence selection; perturbation stress test; non-crash split	§IV-A, §VI-C, §VI-D
R8	Benchmark with issues, logs, code, ground-truth faulty regions	LogRootBench (public, regenerable)	§III
R9	Beat the strongest available baseline on Recall@K, MRR, Acc@K	Main comparison with paired significance tests	§V, §VI-A

III. LOGROOTBENCH: BENCHMARK CONSTRUCTION

LogRootBench is built automatically from public sources: Apache Jira issues, the Apache git mirrors and the Pathidea dataset [12]. We release regeneration scripts and derived labels rather than re-hosting raw issue text.

A. Sources

Pathidea data. Chen et al. [11] collected log-bearing bug reports from eight Apache Java systems (ActiveMQ, Hadoop Common, HDFS, MapReduce, YARN, Hive, Storm and ZooKeeper). The public release [12] contains 1,275 `ISSUE@fixSHA` pairs with class-level ground truth, plus the full Jira records. Its ground truth is too coarse for the challenge, but the fix SHAs allow us to recover the exact changed methods. Filtering out 337 reports without extractable log or stack evidence, 41 commits touching only tests, newly added methods or class-level declarations, and 2 reports empty after leakage masking yields 895 pre-2019 instances (created June 2006–December 2018).

Post-cutoff issues. Public issues from before 2019 are likely in LLM training data. We therefore mine Apache Jira for resolved, fixed bugs created on or after 2025-01-01 across nine Apache Jira projects spanning six git repositories, adding Cassandra, HBase and Kafka alongside Hadoop Common, HDFS, MapReduce, YARN, Hive and ZooKeeper (11 Apache Java systems across eight repositories in total). From 1,941 mined reports (1,777 linked to git commits), we retain issues with at least two runtime log lines—or at least one runtime log line plus three stack-trace frames—and a non-empty pre-fix method ground truth, yielding 107 post-2025 instances (HBase 32, Cassandra 23, Hive 19, Kafka 16, YARN 7, HDFS 5, Hadoop 2, ZooKeeper 2, MapReduce 1; created January 2025–September 2026). Together, LogRootBench comprises 1,002 instances.

B. Snapshots and method-level ground truth

For each issue, the snapshot S is the first parent (`fix^1`) of the fixing commit. We parse the pre-fix and post-fix versions of each changed Java file with tree-sitter-java: every deleted line maps to its innermost enclosing method or constructor in S, and every added line maps to its post-fix method and is matched back to S by $(\text{class}, \text{name}, \text{params})$ (or by $(\text{class}, \text{name})$ when a single overload changed signature). Newly added methods (present in 301/1,002 fixes) and class-level edits outside methods are recorded in metadata and excluded from G, as they do not exist as methods in S. We drop test files, generated sources, non-Java files, and methods whose pre- and post-fix token streams differ only in comments or whitespace; issues whose remaining G is empty are removed. When multiple commits mention an issue key (695/1,002 instances, typically branch backports), we verify that the chosen commit subject leads with the issue key—resolving nine Pathidea entries whose recorded SHA belonged to a follow-up ticket to the earliest non-revert fix commit leading with that key—and record sibling commits in `extra_fix_shas`. Method identifiers have the form `file::Class.method(params)`, with nested classes written as `Outer.Inner` and constructors as `<init>`. The module label of each method follows the definition in Sec. II.

C. Input construction and leakage masking

The issue input is the title and description, taken from the issue as filed rather than the comment thread written during or after the fix. Recent Apache issues often contain an explicit "Root Cause" or "Proposed fix" section that names the faulty method, which would make localization trivial. We therefore (a) strip sections headed *root cause*, *fix*, *patch*, *solution* or similar, as well as inline fix-cue sentences, embedded diffs and pull-request/commit URLs; and (b) mask code-syntax occurrences of ground-truth method names outside log and stack-trace lines. Names inside logs are kept, because they are genuine runtime evidence. Across LogRootBench, 284 instances (28.3%; 498 masked spans: 230/895 [25.7%] in Pathidea and 54/107 [50.5%] in post-2025) required masking. Log evidence is extracted from {`noformat`}/{`code`} blocks and description text; each line is classified as a runtime log line (timestamp/level pattern), a stack frame (`at pkg.Class.method(File.java:N)`) or exception/other line, and receives a stable identifier (`L1`, `L2`, ...).

D. Automatic labels

Silver log-to-statement labels. For every logging call in S (e.g., `LOG.warn(...)`), we compile a regular expression from its format string. Literal parts are kept, and placeholders ({}, `%s`, concatenated expressions) become wildcards. A runtime log line is labelled with the method whose template matches it with the longest literal coverage (supplemented by explicit `File.java:N` logger locations when present); very short templates are left unlabelled. Across 4,310 runtime log lines in LogRootBench, 2,802 (65.0%) match a prefix logging statement, in 751 of 951 log-bearing instances (2,554/3,907 [65.4%] in Pathidea; 248/403 [61.5%] in post-2025). These silver labels define logging sites and supervise the retrieval-stage evaluation (RQ2). **Crash vs. non-crash.** An instance is a *crash* if L contains stack frames (758 instances, 75.6%) and *non-crash* otherwise (244 instances, 24.4%). **Symptom≠cause.** Combining silver log-statement matches with top in-project stack frames resolves at least one logging site for 920/1,002 instances (91.8%; 82 unanchored instances have symptom≠cause undefined). Of the 920 anchored instances, 524 (57.0%; 52.3% of LogRootBench) satisfy symptom≠cause and 396 satisfy symptom=cause. **Hop distance.** This is the shortest distance (capped at ≥ 3) along the stack trace or static call graph between any resolved logging site and any ground-truth method; it is resolved for 830/1,002 instances (median 1 hop; 396 at hop 0 and 434 at ≥ 1 hop).

E. Noisy and incomplete log variants

To test robustness across severity levels $p \in \{0.0, 0.25, 0.50, 0.75\}$, we derive seeded perturbations of L for the evaluation instances: (a) *drop-decisive*: remove $\mathrm{round}(p \cdot |L|)$ highest-signal diagnostic lines (stack frames, exception headers, `FATAL`/`ERROR` lines and silver-anchored lines, retaining ≥ 1 line); (b) *tail truncation*: retain only the earliest $\max(1, \mathrm{round}((1-p)\cdot|L|))$ lines, simulating truncated bundle capture; (c) *noise injection*: interleave $\mathrm{round}(m \cdot |L|)$ benign (`INFO`/`DEBUG`) lines sampled from other projects in the corpus for $m \in \{0, 1, 2, 4\}$, preserving true-line order; and (d) *retry storms*: insert $k = \mathrm{round}(8p) \in \{0, 2, 4, 6\}$ consecutive timestamp-shifted copies of up to $\lfloor |L|/3 \rfloor$ non-fault lines.

F. Quality control and evaluation subsets

All 1,002 instances pass automated checks of snapshot indexing, ground-truth method existence, evidence presence, label consistency, leakage and cutoff dates. An LLM audit (`agnes-3.0-flash`, temperature 0; test-retest Gwet's AC1 = 0.929) confirmed 979 of 1,000 labelled fix commits as genuine bug fixes. Rather than dropping borderline cases after the freeze, 80 flagged instances (e.g., $|G| > 10$ methods, backport commits) are retained with metadata flags. In a manual check of 30 stratified issues (20 Pathidea, 10 post-2025), with the automatic labels hidden from the reviewer, **[human check: genuine bug fix x/30; GT methods are the root cause y/30 (z partial); logs relevant w/30; remaining leaks v/30]**. Non-LLM methods run on all 952 test instances (845 Pathidea + 107 post-2025). LLM-based methods run on a stratified subset of 400 Pathidea test instances plus all 107 post-2025 instances (507 total). A disjoint development split of 50 Pathidea instances is used for all tuning and configuration choices; Sec. VII discusses how test results informed one design decision. Table II summarizes LogRootBench.

TABLE II
LOGROOTBENCH STATISTICS ACROSS 1,002 INSTANCES (895 PATHIDEA PRE-2019 + 107 POST-2025). S≠C COUNTS ISSUES WHERE NO RESOLVED LOGGING ANCHOR IS IN G (82 UNANCHORED ISSUES OMITTED). GT METH. AND LOG LINES REPORT MEDIAN / MEAN PER ISSUE.

Project	#Issues	Crash	Non-crash	S≠C	GT meth.	Log lines
ActiveMQ	76	51	25	43	2.0 / 4.0	24.5 / 31.8
Hadoop	175	123	52	87	2.0 / 4.4	13.0 / 20.4
HDFS	144	103	41	79	2.0 / 3.7	11.5 / 17.8
MapReduce	97	74	23	56	2.0 / 2.7	16.0 / 20.9
YARN	168	127	41	81	1.0 / 3.9	13.0 / 20.7
Hive	158	143	15	79	1.5 / 4.3	30.0 / 34.4
Storm	56	41	15	29	1.0 / 3.6	22.0 / 32.1
ZooKeeper	21	14	7	11	1.0 / 4.6	9.0 / 14.8
Post-2025	107	82	25	59	1.0 / 3.9	20.0 / 28.2
Total	1,002	758	244	524	2.0 / 3.9	17.0 / 24.5

IV. LOG2ROOT: APPROACH

Log2Root (Fig. 1) turns an instance (I, L, S) into the three required outputs in five stages. Stages 1–3 are static and deterministic and use no LLM: they compress noisy logs into anchored evidence and narrow millions of lines of Java code to a ranked pool of candidate methods connected to those anchors. A top-down, issue-guided localizer adds candidates that log anchoring cannot reach. Stages 4–5 execute three structured LLM calls (*Tracer* → *Debugger* → *Reviewer*) interleaved with deterministic grounding checks.

A. Stage 1: Evidence selection

Stack-trace blocks (with `Caused by` chains) are deduplicated by frame signature; other log lines are clustered with a Drain-style parser [17], [18] (depth 4, similarity 0.5) and collapsed to their earliest occurrence with multiplicity c. Each item is scored by severity (`ERROR`/`FATAL` 1.0 down to `TRACE` 0.1), stack or exception presence, proximity to the first anomaly and an exact logging-statement match, minus penalties for boilerplate and repetition ($-0.1 \ln c$). We keep all stack blocks and the top 15 clusters in chronological order, with `L#` identifiers for citation, and map each to up to three non-test *anchor* methods through compiled logging/`throw` templates and resolved in-project stack frames (boosting the deepest `Caused by` frame).

B. Stage 2: Hybrid log-to-code retrieval

For each snapshot S we parse all non-test Java files with `tree-sitter-java`, extract every logging and `throw` statement's compiled message template (resolving string builders and constants) and build a static call graph (receiver typing, class-hierarchy dispatch, name/arity fallback). Three channels rank non-test methods: (a) *template/anchor matching*

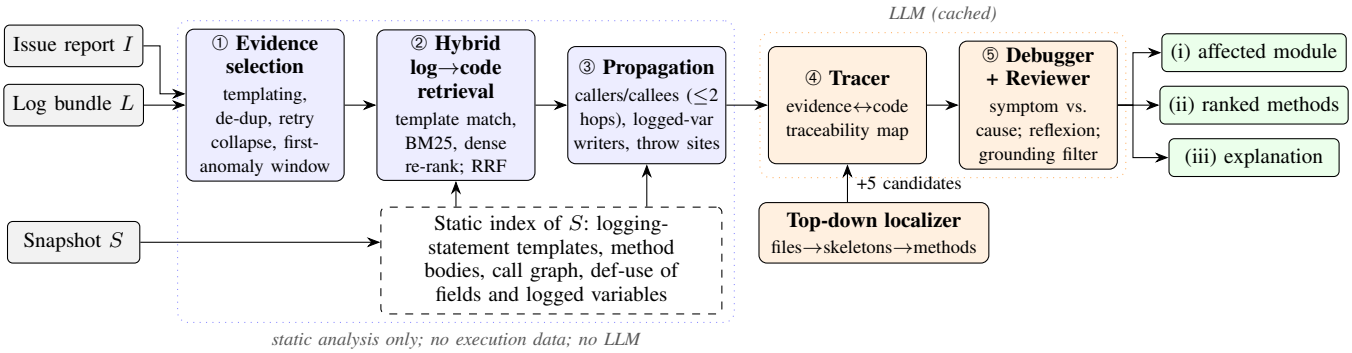


Fig. 1. Overview of Log2Root. Stages ①–③ are static and deterministic. A top-down, issue-guided localizer (three LLM calls) adds five candidates to the top 15 static ones. Stages ④–⑤ use structured LLM reasoning whose outputs are checked against the input (grounding filter).

sums per-line anchor weights from Stage 1; it is exact when the emitting statement exists but brittle for dynamically assembled messages; (b) *BM25* [19] ($k_1 = 1.2$, $b = 0.75$) uses a query of masked issue text, evidence messages, exception types and up to 12 stack-frame identifiers; (c) *dense re-ranking* encodes the query and the top 50 BM25 plus top 20 anchored methods with `SweRankEmbed-Small` [15] on CPU. Rankings are fused by reciprocal-rank fusion [20],

$$s_{\text{ret}}(m) = \sum_{r \in \{\text{tpl},\text{bm25},\text{dense}\}} \frac{1}{k + \text{rank}_r(m)}, \qquad k = 60. \tag{1}$$

C. Stage 3: Propagation (symptom → cause) and top-down expansion

A logging site records where a failure surfaced; the defect often lies upstream. From the top 15 anchors we expand a pool (with the top 50 of s_{ret}, capped at 150 methods) along (a) callers and callees up to 2 hops, boosting callees inside the `try` block of a logging `catch`; (b) writers of fields referenced in a matched log statement or its `if`-guard; and (c) methods throwing logged exception types. Each candidate is scored linearly, $s(m) = \mathbf{w}^\top \boldsymbol{\phi}(m)$, over normalized channel, propagation and structural features (hop distance, evidence coverage, issue overlap, logging-site indicator). The weights are fitted by cross-validated coordinate ascent on `dev` only ($w_{\text{tpl}} = 1.0$, $w_{\text{bm25}} = w_{\text{dense}} = 0.5$, $w_{\text{callee}} = 0.25$, $w_{\text{rrf}} = -0.25$). The top 50 form the static ranking (LOG2ROOT-static) and fallback pool.

Top-down candidate expansion. Log-anchored retrieval works bottom-up from the emitting code, so it misses defects far from every logged or stack-traced method; in our error analysis of an initial version, most top-5 misses had no ground-truth method among the 20 LLM candidates (Sec. VI-B). We therefore add a top-down, issue-guided localizer in the style of Agentless [14] that navigates the repository instead of the log anchors: one LLM call selects five files from a pruned repository tree, a second selects methods from their skeletons, and a third ranks the selected bodies. The $N = 20$ LLM candidates are the top 15 of $s(m)$ plus the 5 best top-down methods not already among them. N and the 15+5 split were chosen on `dev` only. Candidates carry no tag revealing their source.

D. Stages 4–5: Tracer, Debugger, Reviewer and grounding

Each candidate body is compressed with `tree-sitter-java` (keeping signatures, logging/`throw`/`catch` statements, control-flow headers and evidence-matching lines) to fit input caps of 11,000 tokens, and the 20 candidates get seeded shuffled IDs (`C01`–`C20`) to counter position bias [21]. *Tracer* receives the masked issue, the `L#` evidence and all candidates and outputs a traceability map that classifies each log item (`PRIMARY_FAILURE`, `PROPAGATED_SYMPTOM`, `CONTEXT`, `NOISE`) and links it to emitting and causally implicated candidates, adapting the Tracer/Debugger split of Birudugadda et al. [22] from dynamic coverage over agent logs to static retrieval over production logs. *Debugger* assigns causal roles (`ROOT_CAUSE`, `CONTRIBUTOR`, `SYMPTOM_SITE`, `UNRELATED`), separating reporting sites from upstream origins, and drafts a ranking, module and cited explanation. A deterministic check flags unknown IDs, citations of `NOISE` lines and top-1 symptom-site picks when upstream candidates exist, and *Reviewer* performs one reflexion round [2], [23] over the draft and these warnings. A final deterministic filter maps IDs back to verified snapshot methods, strips unknown method and log identifiers, fills the ranking up to 50 from the pool, derives files by order-preserving deduplication, validates modules, and rewrites `Cxx` tokens in the explanation to `Class.method`. Every cited line and method is thus verifiable in (L, S) by construction.

V. EXPERIMENTAL SETUP

A. Research questions

RQ1 Effectiveness. How accurately does Log2Root localize root causes at module, file and method level, compared with the strongest available baselines, on the full benchmark, on recent (post-2025) issues and across LLM backbones?

RQ2 Log-to-code retrieval. How accurately does each retriever (template, BM25, dense, hybrid) map log evidence

to the emitting code, and how much does retrieval quality bound end-to-end localization?

RQ3 **Beyond symptoms.** Does Log2Root localize root causes that differ from the logging site, on the symptom≠cause and non-crash splits and as a function of hop distance?

RQ4 **Ablation and robustness.** What does each stage contribute, and how does accuracy degrade under noisy or incomplete logs?

RQ5 **Explanations and practicality.** Are explanations grounded and faithful, and what does Log2Root cost per issue?

B. Baselines

Table III lists the baselines. The *strongest available baseline* is the best baseline on each metric. To isolate localization design from the backbone, all LLM-based baselines share Log2Root's model, temperature, token cap, seeded shuffling, grounding filter and response cache. Two of the closest techniques have no public tool code: no implementation accompanies the Pathidea data [12], and the COCA repository contains no code or data at the time of writing. We therefore re-implement them from their papers (*-style*); our Pathidea re-implementation (800-token VSM segments, logged-class weight $\alpha = 0.1$, 4-hop reconstructed path weight $\beta = 0.2$, plus a *Pathidea→BM25* variant for intra-file method ranking) is verified against the released per-issue VSM scores and paths. Original LLM4FL [2], AgentFL [3] and FaR-Loc [5] require failing tests and dynamic coverage and cannot run in their original form (R1); to compare directly against LLM4FL's multi-agent reflexion design, we implement *LLM4FL-log*, an execution-free log-only adaptation of its Tester→Debugger→Reviewer loop [2] that replaces spectrum coverage with resolved stack/log-anchor seeds, 1-hop static callers/callees and BM25 top-30 methods.

TABLE III
BASELINES. GRAN. = NATIVE OUTPUT GRANULARITY (F = FILE, M = METHOD, MOD = MODULE).

Baseline	Gran.	Description
BM25	F, M	Issue + logs as query over files/methods (k_1=1.2, b=0.75) [19]; IR lower bound
Pathidea-style	F	VSM + logged-class boost + 4-hop log-reconstructed path boost [11]; strongest log-aware IR baseline
LogStmt-only	M	Rank resolved evidence logging sites (deepest stack tops, severity-ordered template matches); *symptom* baseline
SweRank-Embed	M	Trained `SweRankEmbed-Small` [15] cosine re-ranking over BM25 + symptom pool on CPU
LLM+RAG	M	BM25 top-20 shuffled methods → single-call LLM ranking; backbone control
Agentless-style	F, M	3-call hierarchical repo-tree + BM25 → top-5 Java skeletons → method ranking [14]
COCA-style	M, Mod	Logging-source anchors → 2-hop static paths + historical issues → 2-call LLM RCA [13]
LLM4FL-log	M, Mod	Log-only adaptation of LLM4FL's Tester→Debugger→Reviewer loop [2] (3–4 calls, no coverage)

C. Metrics

For an instance with ground truth G and ranked list R, let $\mathrm{rank}(g)$ be the 1-based position of the first entry of R that matches $g \in G$, and $R_{\le K}$ the top-K entries. We report

$$Acc@K = \mathbb{1}\big[\exists g \in G:\ \mathrm{rank}(g) \le K\big], \tag{2}$$

$$Recall@K = |\{g \in G:\ \mathrm{rank}(g) \le K\}| \,/\, |G|, \tag{3}$$

$$MRR = 1/\min_{g \in G} \mathrm{rank}(g) \quad (0 \text{ if no } g \text{ is ranked}), \tag{4}$$

averaged over instances, with $K \in \{1, 3, 5\}$ for Acc@K and $K \in \{5, 10\}$ for Recall@K. **Matching rule.** A predicted method matches a ground-truth method if file, class and method name agree. Parameter lists are ignored, so overloads count as the same method. Files match by path and modules by module path. When a system ranks only methods, its file and module rankings are derived by order-preserving deduplication of the methods' files and modules. **Retrieval-stage metrics (RQ2).** Each retrieval channel's ranking is scored against the ground-truth methods (Acc@1, MRR, Hit@k). For the log-to-statement step we report agreement with the logger class named in each log line, which the matcher does not use; we do not use the silver labels for this, because they were produced by a similar template matcher. **Explanation metrics (RQ5).** An explanation is *valid* if it cites at least one log line and one method and every cited line exists in L and every cited method exists in S (checked with `git` and tree-sitter for all systems alike). *Faithfulness* and *oracle mechanistic match* are scored on a stratified sample of 50 test issues (balanced across crash/non-crash, symptom≠cause and source) by an LLM judge (`gpt-4.1`, temperature 0, a model no system uses) that receives the masked issue, the logs, the ground-truth code and five explanations per issue (Log2Root, its variant without top-down expansion and Agentless-style on `gpt-5-mini`; the latter two also on `agnes-3.0-flash`), anonymized and shuffled per issue. LLM judges can be biased, so we treat these scores as indicative and complement them with a human rating (Sec. VI-E). **Significance.** We compare Log2Root with each baseline using a two-sided paired Wilcoxon signed-rank test on continuous per-instance metrics (MRR, Recall@K) and a two-sided exact McNemar test on binary hits (Acc@K), with Holm-Bonferroni adjustment over the baselines of a table (per metric), and paired percentile bootstrap 95% confidence intervals (10,000 resamples) for differences. Comparisons on splits and between Log2Root and its own ablations are reported with unadjusted p-values.

D. Implementation

Log2Root and all baselines are implemented in Python 3.13 using `tree-sitter-java` 0.23 and `sentence-transformers` on a CPU-only machine (8 vCPUs, 31 GB RAM, no GPU). Prompts were developed on `dev` with `agnes-3.0-flash` and then frozen. The main backbone, `gpt-5-mini` (low reasoning effort), was

selected on `dev` among four candidates (`gpt-4.1-mini`, `gpt-5-mini`, `gpt-oss-120b`, `agnes-3.0-flash`); all LLM systems run on it for the 507 evaluation issues. As a second backbone, the open-weight `gpt-oss-120b` [24], which can be deployed on premises for proprietary logs, runs Log2Root, its variant without top-down expansion and Agentless-style on 160 evaluation issues (75 Pathidea, 85 post-2025): those of a stratified 200-issue sample that the baselines completed within the provider's free-tier quota. Earlier runs of the variant without expansion and of all baselines on `agnes-3.0-flash` (507 issues) and `agnes-2.5-flash` (100) are further cross-model checks. Temperature 0 and seed 42 are used where supported (not by `gpt-5-mini`). All calls were made in November 2026 (about $32 for `gpt-5-mini` and `gpt-4.1`; free-tier quotas otherwise) and are cached by SHA-256 hash for offline replay.

VI. Results

A. RQ1: Effectiveness

Table IV reports all three output levels. Without any LLM call, Log2Root-static beats every non-LLM baseline at method Acc@5 and MRR; on all 952 test issues it reaches 0.624 method Acc@5 and 0.438 MRR, against 0.537 and 0.396 for the strongest non-LLM baseline (LogStmt-only; $p_{\text{Holm}} < 0.01$). With the LLM stages, Log2Root has the best method-level value in every column (Acc@1/Acc@5/MRR 0.515/0.720/0.606). At module level, however, Agentless-style is better (0.919 vs. 0.822 Acc@1): with `gpt-5-mini`, Log2Root's explicit module prediction is weaker than its method ranking, and taking the module of its top-ranked method would give 0.911 (a post-hoc diagnostic, not used in the table). Against the strongest baseline, Agentless-style, the gain is significant for method Acc@5 ($+0.057$, 95% CI $[+0.022, +0.095]$, $p_{\text{Holm}} = 0.003$) and, marginally, MRR ($+0.042$, $p_{\text{Holm}} = 0.045$), but not for method Acc@1 ($+0.024$, $p = 0.285$). Against LLM4FL-log, all three are significant ($p_{\text{Holm}} \leq 0.002$). On the 107 post-2025 issues, Log2Root and Agentless-style tie at Acc@1 (0.505 vs. 0.505) and Log2Root leads at Acc@5 (0.766 vs. 0.682, $p = 0.078$).

Table V repeats the comparison on other models. On the open-weight `gpt-oss-120b` ($n = 160$), the full method reaches 0.450 Acc@1, 0.731 Acc@5 and 0.578 MRR, against 0.500, 0.731 and 0.591 for Agentless-style (paired $p \geq 0.24$) and 0.444, 0.688 and 0.548 without top-down expansion. Without top-down expansion, the pipeline is level with Agentless-style: ahead on `gpt-5-mini` and `agnes-3.0-flash` (significant after Holm correction only at Acc@5 on `agnes-3.0-flash`, the backbone its prompts were tuned on) and behind, not significantly, on `gpt-oss-120b` and `agnes-2.5-flash`. Top-down expansion raises all three metrics on both backbones on which we ran it (significantly only Acc@1 on `gpt-5-mini`), but yields a significant lead over Agentless-style only on `gpt-5-mini`; on `gpt-oss-120b`, Log2Root ties at Acc@5 and trails at Acc@1 and MRR.

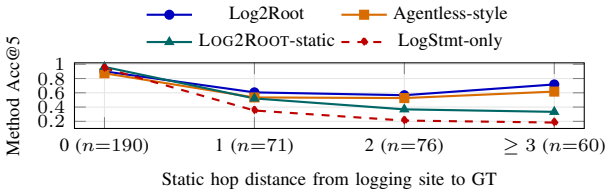


Fig. 2. RQ3: method Acc@5 vs. static hop distance (397 hop-resolved issues).

Answer to RQ1: The static stage alone beats all non-LLM baselines at method Acc@5 and MRR. On `gpt-5-mini`, the full Log2Root is the most accurate system at method level and significantly better than the strongest LLM baseline at top-5 (marginally at MRR), but not at top-1, and weaker at module level; on `gpt-oss-120b` it only matches that baseline.

B. RQ2: Log-to-code retrieval

Table VI evaluates the static channels separately. Template matching is precise when the emitting statement is found but covers few root causes (Hit@20 0.513), because upstream and non-crash defects rarely log at the fault site. Dense retrieval is the strongest single channel, and fusion plus Stage-3 scoring gives the best ranking (0.438 MRR, Hit@20 0.774). As an independent check of the log-to-statement step (not reusing the silver labels, which come from a similar matcher), the method matched to a log line without using its logger name lies in the logger's class in 79.8% of 2,530 lines.

Retrieval bounds the whole pipeline (Table VII). Without top-down expansion, 117 of the 152 issues without a GT method in the top 5 (77%) fail before the LLM sees the defective method. Replacing the five lowest static candidates with five top-down candidates raises the share of issues whose 20 LLM candidates contain a GT method from 0.769 to 0.821, and cuts outright retrieval misses from 87 to 62. The remaining retrieval misses are dominated by symptom≠cause (77%) and over-represent non-crash issues (42% vs. 29% overall). Some recovered issues are then ranked below the top 5, so rank misses grow from 35 to 51.

Answer to RQ2: Dense code retrieval is the most useful log-to-code channel and fusion helps, but log-anchored retrieval alone leaves 77% of top-5 misses unrecoverable; top-down candidates reduce this share to 64%.

C. RQ3: Beyond symptoms

Ranking logging sites works when the symptom site is the cause: LogStmt-only reaches 0.668 Acc@1 on S=C issues but only 0.012 on the 254 S≠C issues (Table VIII), and its Acc@5 falls from 0.947 at hop 0 to 0.183 at ≥ 3 hops (Fig. 2). LLM reasoning over retrieved candidates closes much of this gap: on S≠C, Log2Root reaches 0.354 Acc@1 and 0.598 Acc@5, significantly above LLM4FL-log (0.252 Acc@1, $p = 0.001$) and above Agentless-style at top-5 (0.528 Acc@5, $p = 0.013$) but not at top-1 (0.335, $p = 0.597$). The same holds on non-crash issues: Log2Root leads Agentless-style at top-5 (0.683 vs. 0.607, $p = 0.035$), and its top-1 lead (0.538 vs. 0.476, $p =$

TABLE IV

RQ1: LOCALIZATION ON THE 507-ISSUE LLM EVALUATION SUBSET (400 STRATIFIED PATHIDEA + 107 POST-2025 ISSUES; ALL LLM SYSTEMS USE `GPT-5-MINI`). BEST PER COLUMN IN BOLD. $^{\dagger}/^{\ddagger}$: LOG2ROOT IS SIGNIFICANTLY BETTER THAN THAT BASELINE AT $p_{\text{Holm}} < 0.05/< 0.01$ (PAIRED WILCOXON ON MRR/RECALL@K, EXACT MCNEMAR ON ACC@K; HOLM CORRECTION PER COLUMN OVER THE 10 BASELINE ROWS, I.E., ALL ROWS EXCEPT THE TWO LOG2ROOT ROWS). "W/O TOP-DOWN" IS OUR ABLATION WITHOUT TOP-DOWN CANDIDATE EXPANSION.

System	Module		File			Method					
	Acc@1	Acc@3	Acc@1	Acc@5	MRR	Acc@1	Acc@3	Acc@5	Recall@5	Recall@10	MRR
BM25	0.767	0.917†	0.357‡	0.613‡	0.472‡	0.170‡	0.256‡	0.310‡	0.194‡	0.266‡	0.241‡
Pathidea-style	0.753‡	0.929	0.389‡	0.694‡	0.526‡	–	–	–	–	–	–
Pathidea→BM25	0.753‡	0.929	0.389‡	0.694‡	0.526‡	0.168‡	0.241‡	0.274‡	0.180‡	0.250‡	0.227‡
LogStmt-only	0.736‡	0.947	0.391‡	0.742‡	0.540‡	0.278‡	0.460‡	0.531‡	0.359‡	0.467‡	0.398‡
SweRank-Embed	0.860	0.959	0.489‡	0.771‡	0.614‡	0.270‡	0.422‡	0.515‡	0.353‡	0.451‡	0.382‡
LOG2ROOT-static	0.799	0.970	0.450‡	0.799‡	0.602‡	0.308‡	0.536‡	0.621‡	0.440‡	0.537‡	0.445‡
LLM+RAG	0.838	0.927	0.521‡	0.680‡	0.591‡	0.310‡	0.379‡	0.424‡	0.300‡	0.335‡	0.359‡
COCA-style	0.811	0.959	0.491‡	0.724‡	0.598‡	0.341‡	0.465‡	0.517‡	0.363‡	0.414‡	0.424‡
LLM4FL-log	0.880	0.968	0.598‡	0.821‡	0.694‡	0.442‡	0.600†	0.645‡	0.469‡	0.528‡	0.535‡
Agentless-style	**0.919**	**0.978**	0.653	**0.876**	0.743	0.491	0.621	0.663‡	0.487‡	0.529‡	0.564†
Log2Root w/o top-down	0.826	0.957	0.625	0.848	0.725	0.477	0.629	0.700	0.509	0.566	0.571
Log2Root	0.822	0.961	**0.673**	0.872	**0.761**	**0.515**	**0.657**	**0.720**	**0.530**	**0.610**	**0.606**

TABLE V

RQ1 ACROSS BACKBONES (METHOD LEVEL, PAIRED ISSUES). †: LOG2ROOT BETTER THAN AGENTLESS-STYLE AT PAIRED, UNADJUSTED $p < 0.05$. PROMPTS WERE TUNED ON `DEV` WITH `AGNES-3.0-FLASH`; THE FULL METHOD WAS NOT RUN ON THE `AGNES` MODELS.

Backbone	n	System	Acc@1	Acc@5	MRR
`gpt-5-mini`	507	Log2Root	0.515	0.720†	0.606†
		w/o top-down	0.477	0.700	0.571
		Agentless-style	0.491	0.663	0.564
`gpt-oss-120b`	160	Log2Root	0.450	0.731	0.578
		w/o top-down	0.444	0.688	0.548
		Agentless-style	0.500	0.731	0.591
`agnes-3.0-flash`	507	w/o top-down	0.521	0.720	0.606
		Agentless-style	0.503	0.663	0.576
`agnes-2.5-flash`	100	w/o top-down	0.430	0.680	0.539
		Agentless-style	0.500	0.710	0.592

TABLE VI

RQ2: EACH RETRIEVAL CHANNEL ON ITS OWN, AGAINST THE GROUND-TRUTH METHODS (ALL 952 TEST ISSUES). HIT@k: A GT METHOD IS IN THE TOP k.

Channel / stage	Acc@1	MRR	Hit@20	Hit@50
Template / anchor matching	0.223	0.331	0.513	0.513
BM25	0.230	0.340	0.663	0.754
Dense (`SweRankEmbed-Small`)	0.269	0.392	0.732	0.805
Hybrid (RRF)	0.296	0.431	0.769	0.807
+ propagation & scoring (Stage 3)	0.295	0.438	0.774	0.814

0.163) is not significant. The characteristic error the challenge describes persists: on S≠C issues that Log2Root misses at top-1, its first-ranked method is a logging or stack site in 48% of cases.

Answer to RQ3: Logging-site ranking fails almost completely when symptom≠cause; LLM reasoning over the candidates recovers part of the gap, for Log2Root and Agentless-style alike at top-1; Log2Root is ahead at top-5.

TABLE VII

WHERE EACH ISSUE IS DECIDED (507 ISSUES, `GPT-5-MINI`), WITHOUT AND WITH TOP-DOWN EXPANSION.

Outcome	w/o top-down	Log2Root
GT method ranked first	242	261
Best GT rank 2–5	113	104
Miss: GT among the 20 LLM candidates, rank > 5	35	51
Miss: GT in the candidate pool, not in the 20	30	29
Miss: GT not in the candidate pool	87	62

TABLE VIII

RQ3: METHOD ACC@1 / MRR BY REGIME ($n = 507$: 190 S=C, 254 S≠C, 63 WITHOUT A RESOLVED LOGGING SITE; 362 CRASH, 145 NON-CRASH; LLM SYSTEMS ON `GPT-5-MINI`).

System	S=C	S≠C	Crash	Non-crash
LogStmt-only	0.668 / 0.799	0.012 / 0.123	0.282 / 0.407	0.269 / 0.376
LOG2ROOT-static	0.658 / 0.795	0.047 / 0.193	0.293 / 0.445	0.345 / 0.445
COCA-style	0.679 / 0.764	0.130 / 0.214	0.345 / 0.433	0.331 / 0.403
LLM4FL-log	0.705 / 0.802	0.252 / 0.348	0.420 / 0.525	0.497 / 0.561
Agentless-style	0.705 / 0.777	0.335 / 0.415	0.497 / 0.574	0.476 / 0.538
Log2Root w/o top-down	**0.747 / 0.828**	0.303 / 0.402	0.478 / 0.580	0.476 / 0.550
Log2Root	0.726 / 0.809	**0.354 / 0.456**	**0.506 / 0.605**	**0.538 / 0.607**

D. RQ4: Ablation and robustness

Top-down expansion gives the largest gain among the LLM-side design choices (Table IX), although only its top-1 effect is significant: removing it costs 3.7 pp Acc@1 ($p = 0.034$), 2.0 pp Acc@5 ($p = 0.289$) and 0.034 MRR ($p = 0.364$). The two candidate sources are complementary rather than redundant: the expanded pool also beats Agentless-style itself, which ranks only top-down candidates (Table IV). Reviewer reflexion changes almost nothing ($p = 0.084$ on MRR), and on `agnes-3.0-flash` none of the other LLM-stage ablations of the version without expansion (removing Tracer, shuffling, or both Tracer and Reviewer) differs significantly from it (all $p \geq 0.06$). The LLM ranking itself is what turns candidates

TABLE IX
RQ4: ABLATIONS ON THE 507-ISSUE SUBSET (METHOD LEVEL, `GPT-5-MINI`). p: PAIRED WILCOXON ON MRR VS. THE FULL METHOD.

Variant	Acc@1	Acc@5	MRR	p
Log2Root (full)	0.515	0.720	0.606	
– Reviewer	0.513	0.716	0.604	0.08
– top-down expansion	0.477	0.700	0.571	0.36
LOG2ROOT-static (no LLM)	0.308	0.621	0.445	
– evidence selection	0.300	0.617	0.440	
– template matching	0.306	0.584	0.428	
– dense re-ranking	0.282	0.521	0.399	
– static propagation	0.302	0.627	0.440	

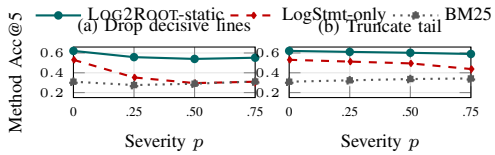


Fig. 3. RQ4: method Acc@5 of the static systems when decisive lines are dropped or the tail is truncated, severity $p \in \{0, .25, .50, .75\}$ ($n = 507$).

into answers: relative to the static ranking it moves 150 issues to a correct top-1 and loses 45. In the static stage, dense re-ranking (10.1 pp Acc@5) and template matching (3.7 pp) contribute most; evidence selection and static propagation have no measurable aggregate effect. Under perturbation (Fig. 3), the static stage degrades gracefully: with 75% of the decisive lines removed it keeps 0.552 Acc@5 (from 0.621), while LogStmt-only falls from 0.531 to 0.310; under $4\times$ cross-issue noise and retry storms it keeps 0.584 and 0.623.

Answer to RQ4: Combining log-anchored and top-down candidates gives the largest gain; dense retrieval and the LLM ranking call carry the rest, while the Tracer/Reviewer stages and static propagation give no significant gain. The static stage is robust to noisy and incomplete logs.

E. RQ5: Explanations and practicality

Explanations. On all 507 issues, every system cites only log lines and methods that exist in the issue and snapshot; Log2Root guarantees this by construction through its deterministic post-filter. On 50 stratified issues, an independent LLM judge (`gpt-4.1`, not used by any system) saw the masked issue, the logs, the ground-truth code and five anonymized explanations per issue in shuffled order. It rated the explanation faithful to the evidence for 100% of issues with Log2Root, 98% with Agentless-style and 98% without top-down expansion, and both faithful and consistent with the real defect for 96%, 92% and 94%; the differences are within one or two issues. On identical `agnes-3.0-flash` explanations it agreed with an earlier same-model judge on 95%/89% of 100 ratings. The human check of the explanations is pending (**[sheet 2]**).

Cost and memorization. Log2Root makes six LLM calls per issue (three top-down, three reasoning) and costs about $0.023 per issue on `gpt-5-mini` at list price (39k tokens), against $0.008 for Agentless-style and $0.015 without expansion; its static stage needs a median of 11 s per issue on a CPU. Asked for the fixed methods given only an issue key, `gpt-5-mini` claimed to know 0% of the 507 issues and offered no guess; `agnes-3.0-flash`, which did guess, recalled no exact file path, no `Class.method`, and a file basename for only 1.2% (1.5% pre-2019, 0.0% post-2025), so memorized issue knowledge is unlikely to explain the results.

Case study (non-crash, symptom≠cause). In `HADOOP-13487`, Hadoop KMS never removes expired delegation tokens from ZooKeeper. The only attached log line reports that the token-remover thread started, and the fix is in `ZKDelegationTokenSecretManager.startThreads`, which did not load existing tokens at startup. None of LOG2ROOT-static, Log2Root without expansion, LogStmt-only, SweRank-Embed, COCA-style, or LLM4FL-log has it in its top 50. Agentless-style finds it (rank 3) but blames a token-reading method. With the method among its candidates, Log2Root ranks it first, using the absent removal messages as evidence: *"Because ZKDelegationTokenSecretManager.removeStoredToken's deletion logs are absent, the most likely cause is tokens never being loaded into currentTokens at startup [...]"* Expansion also has a cost: in `HDFS-17847` the defective method sat at static rank 19, dropped out of the 15 retained static candidates, and fell from rank 5 to rank 24.

VII. DISCUSSION AND THREATS TO VALIDITY

Implications for practice. The static stage alone routes 79.9% of the 507 issues to the right module (97.0% in the top 3) in about 11 s on a CPU, though SweRank-Embed routes better (86.0%); with `gpt-5-mini`, the module Log2Root names explicitly is less accurate than its method ranking, which a future version can fix by deriving the module from the top-ranked method. When the logging site is the cause, LogStmt-only already ranks a defective method first in 66.8% of issues; LLM reasoning pays off on the symptom≠cause majority. Log-anchored and top-down retrieval miss different root causes, so a practical tool should use both; even combined, Log2Root and Agentless-style succeed on partly different issues (either is correct at top-1 for 60.7%, at top-5 for 78.1%), so candidate selection remains the main lever.

Design history and test-set reuse. We designed the top-down expansion after the error analysis of an initial version, which had been evaluated on the test issues, showed that most top-5 misses were retrieval misses. Before that, we tried a rank-level fusion of the initial version's and Agentless-style's outputs on the test issues and discarded it, as it fell between the two. To limit test-set adaptation, the expansion reuses an existing baseline's localizer unchanged, its configuration (15+5 candidates) and the backbone were chosen on `dev` only, and the frozen design was run on the test issues once. We report the initial version as the "w/o top-down" ablation on every backbone. Some optimism from this sequence cannot be excluded.

TABLE X
POSITIONING. EXEC.-FREE: NO TESTS, COVERAGE OR TRACES NEEDED. LOGS: LOG SEMANTICS USED FOR LOCALIZATION. GRAN.: OUTPUT GRANULARITY. EXPL.: EXPLANATION. PUBLIC: CODE AND DATA AVAILABLE. ○ = PARTIAL (PUBLIC: DATA ONLY); ? = UNKNOWN; *CRASH ONLY.

Work	Exec.-free	Logs	Gran.	Expl.	Public
Pathidea [11]	✓	✓	file	–	○
COCA [13]	✓	✓	component	✓	–
ErrorPrism [25]	✓	✓	error path	○	–
KernelDiag [26]	✓	✓	method*	✓	?
LLM4FL [2]	–	–	method	–	✓
Agentless [14], SweRank [15]	✓	–	function	–	✓
RCAgent [6], RCACopilot [7]	✓	✓	service	✓	–
Log2Root + LogRootBench	✓	✓	mod.+meth.	✓	✓

Backbones and explanation evaluation. Prompts were tuned on `dev` with `agnes-3.0-flash`, and the main runs use `gpt-5-mini`, which does not support temperature 0; cached outputs make our numbers replayable but not bit-reproducible by re-querying. The margin over Agentless-style depends on the backbone: on `gpt-oss-120b` (160 issues) Log2Root ties at Acc@5 and trails at Acc@1 and MRR (not significant). The judge is a different model from every backbone, but LLM judges remain imperfect and our sample is small, so we complement it with a human rating.

Data contamination. Pathidea issues are pre-2019 and public. We (i) evaluate the 107 issues filed in 2025–2026 separately, where Log2Root keeps its accuracy (0.505 Acc@1 / 0.766 Acc@5 vs. 0.518 / 0.708 pre-2019), (ii) strip root-cause sections and mask ground-truth method names in issue text, and (iii) probe models for memorized fixes (Sec. VI-E). Training cutoffs of hosted models are not fully documented, so the 2025–2026 split reduces but does not exclude exposure.

Benchmark validity. Ground truth is the set of methods changed by the fix, which can include incidental edits. Named automated checks pass on all 1,002 issues and 80 flagged issues are kept with flags (60/60 agreement with an independent diff oracle), an LLM audit classifies 979/1,000 fix commits as genuine bug fixes, and a manual check of a stratified sample of 30 issues **[human check result]** (Sec. III-F). The silver log-to-statement labels come from a template matcher similar to Log2Root's, so we do not use them to evaluate Log2Root's retrieval. Static call graphs from `tree-sitter-java` approximate dynamic dispatch. Pathidea, COCA and LLM4FL are re-implementations (no released tool code, or coverage needed); we release all baseline code.

External validity. Logs in LogRootBench are those pasted into bug reports (median 17 lines), not complete production bundles; our noise conditions are synthetic and applied to the static stage only. Industrial telemetry, other languages and proprietary systems remain to be studied.

VIII. RELATED WORK

Log-guided localization with code. Pathidea [11] matches bug-report logs to logged classes and boosts files along static call paths; a companion study examined user-reported logs in bug reports [27]. LogRootBench upgrades Pathidea's public data to method-level ground truth, module labels, symptom≠cause/non-crash splits and post-2025 issues, while Log2Root introduces hybrid, separately evaluated log-to-code retrieval. COCA [13] prompts an LLM over logging sources and reconstructed paths on 106 issues from Yuan et al. [10], but outputs responsible components (including non-code concepts) rather than ranked methods and does not evaluate retrieval separately. LogHound [28] adds static coverage estimates to path reconstruction; ErrorPrism [25] searches multi-hop error paths from log strings; PRAXIS [29] traverses service/program graphs for cloud incidents; KernelDiag [26] builds log-to-code evidence graphs for kernel crashes; and Holmes [30] diagnoses mobile crashes from stacks, logs and thread states. None provides a public method-level benchmark with symptom≠cause labels.

Fault localization with execution data and LLMs. Spectrum- and learning-based FL require failing tests and coverage. SBEST [1] noted that crash reports rarely have fault-triggering tests, and CrashLocator [31] ranks methods from crash stacks. LLM localizers—AutoFL [4], AgentFL [3], LLM4FL [2] and FaR-Loc [5]—all start from failing tests; Log2Root adapts retrieval augmentation and reflexion to static log evidence, using seeded shuffling to counter order bias [21].

Issue and IR bug localization. Agentless [14] localizes hierarchically, and Log2Root reuses such a localizer as its top-down candidate source; unlike generic embedders [32], SweRank [15] trains dedicated issue-to-function retrievers; LocAgent [16], CoSIL [33], OrcaLoca [34] and AutoCodeRover [35] traverse code graphs or combine SBFL on SWE-bench [36], treating logs as plain prose. Classic IRBL—BugLocator [37], BLUiR [38], Locus [39] and BRTracer [40]—scores report-to-file similarity.

Log analysis and AIOps RCA. Log parsing [17], [18], LoFI [9], LogPilot [41] and LogConfigLocalizer [42] extract diagnostic lines or misconfigurations, static logging studies [8] motivate joint log-code semantics, and cloud RCA agents and benchmarks [6], [7], [43]–[45] target services or metrics, not source methods.

IX. CONCLUSION

We contributed LogRootBench, a 1,002-issue public benchmark for execution-free, log-guided root-cause localization; Log2Root, which combines bottom-up log-to-code retrieval, top-down candidate expansion and grounded LLM reasoning; and a study against nine baselines. With `gpt-5-mini`, Log2Root is significantly better than the strongest baseline at method-level top-5 (not at top-1 or module level); on `gpt-oss-120b` it is not. Future work targets module routing and industrial log bundles.

REFERENCES

[1] M. N. Rafi, L. B. S. Pacheco, A. R. Chen, J. Yang, and T.-H. P. Chen, "SBEST: Spectrum-based fault localization without fault-triggering tests," *Empirical Software Engineering*, vol. 31, no. 1, 2025.

[2] M. N. Rafi, D. J. Kim, T.-H. Chen, and S. Wang, "A multi-agent approach to fault localization via graph-based retrieval and reflexion," arXiv preprint arXiv:2409.13642, 2024. [Online]. Available: https://arxiv.org/abs/2409.13642

[3] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "SoapFL: A standard operating procedure for LLM-based method-level fault localization," *IEEE Transactions on Software Engineering*, pp. 1173–1187, 2025, preprint titled "AgentFL: Scaling LLM-based Fault Localization to Project-Level Context", arXiv:2403.16362.

[4] S. Kang, G. An, and S. Yoo, "A quantitative and qualitative evaluation of LLM-based explainable fault localization," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1424–1446, 2024.

[5] X. Shi, Z. Li, and A. R. Chen, "Enhancing LLM-based fault localization with a functionality-aware retrieval-augmented generation framework," arXiv preprint arXiv:2509.20552, 2025. [Online]. Available: https://arxiv.org/abs/2509.20552

[6] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024, pp. 4966–4974.

[7] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, 2024, pp. 674–688.

[8] Y. Li, Y. Huo, R. Zhong, Z. Jiang, J. Liu, J. Huang, J. Gu, P. He, and M. R. Lyu, "Go static: Contextualized logging statement generation," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.12958

[9] J. Huang, Z. Jiang, J. Liu, Y. Huo, J. Gu, Z. Chen, C. Feng, H. Dong, Z. Yang, and M. R. Lyu, "Demystifying and extracting fault-indicating information from logs for failure diagnosis," in *Proceedings of the IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, 2024, pp. 511–522.

[10] W. Yuan, S. Lu, H. Sun, and X. Liu, "How are distributed bugs diagnosed and fixed through system logs?" *Information and Software Technology*, p. 106234, 2020.

[11] A. R. Chen, T.-H. Chen, and S. Wang, "Pathidea: Improving information retrieval-based bug localization by re-constructing execution paths using logs," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2905–2919, 2022.

[12] SPEAR-SE, "Pathidea_Data: Dataset for Pathidea," https://github.com/SPEAR-SE/Pathidea_Data, 2021, accessed Nov. 2026.

[13] Y. Li, Y. Wu, J. Liu, Z. Jiang, Z. Chen, G. Yu, and M. R. Lyu, "COCA: Generative root cause analysis for distributed systems with code knowledge," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1346–1358.

[14] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025, preprint titled "Agentless: Demystifying LLM-based Software Engineering Agents", arXiv:2407.01489.

[15] R. G. Reddy, T. Suresh, J. Doo, Y. Liu, X.-P. Nguyen, Y. Zhou, S. Yavuz, C. Xiong, H. Ji, and S. Joty, "SweRank: Software issue localization with code ranking," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2505.07849

[16] Z. Chen, R. Tang, G. Deng, F. Wu, J. Wu, Z. Jiang, V. Prasanna, A. Cohan, and X. Wang, "LocAgent: Graph-guided LLM agents for code localization," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 8697–8727.

[17] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proceedings of the 2017 IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.

[18] Z. Jiang, J. Liu, J. Huang, Y. Li, Y. Huo, J. Gu, Z. Chen, J. Zhu, and M. R. Lyu, "A large-scale evaluation for log parsing techniques: How far are we?" in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024. [Online]. Available: https://arxiv.org/abs/2308.10828

[19] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, 2009.

[20] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[21] M. N. Rafi, D. J. Kim, T.-H. P. Chen, and S. Wang, "Order matters! An empirical study on large language models' input order bias in software fault localization," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026, pp. 144–155.

[22] S. Birudugadda, S. R. Kondam, N. B. Nakka, E. S. G. Koleti, and S. Mondal, "A localization framework for reasoning faults in LLM-based code agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE), Industry Challenge Track*, 2026, pp. 379–388.

[23] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," arXiv preprint arXiv:2303.11366, 2023. [Online]. Available: https://arxiv.org/abs/2303.11366

[24] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv preprint arXiv:2508.10925, 2025. [Online]. Available: https://arxiv.org/abs/2508.10925

[25] J. Pu, Y. Li, Z. Chen, J. Liu, Z. Jiang, J. Chen, R. Shi, Z. Zheng, and T. Zhang, "ErrorPrism: Reconstructing error propagation paths in cloud service systems," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 3534–3545.

[26] W. Wang, Z. Wang, D. Wang, H. Wang, and J. Chen, "KernelDiag: Agent-based root cause diagnosis for kernel crashes," arXiv preprint arXiv:2607.17722, 2026. [Online]. Available: https://arxiv.org/abs/2607.17722

[27] A. R. Chen, T.-H. P. Chen, and S. Wang, "Demystifying the challenges and benefits of analyzing user-reported logs in bug reports," *Empirical Software Engineering*, vol. 26, no. 1, 2021.

[28] Z. Lin, Y. Feng, J. Chen, T. Zhu, G. Chen, and J. Liu, "LogHound: Coverage-aware path reconstruction for log-based fault localization," in *Proceedings of the 2026 IEEE International Conference on Joint Cloud Computing (JCC)*, 2026, pp. 121–128.

[29] S. Cui, R. Krishna, S. Jha, and R. K. Iyer, "PRAXIS: Integrating program analysis with observability for root-cause analysis," in *Proceedings of the 56th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2026, pp. 52–66.

[30] J. Li, W. Ma, T. Peng, H. Zheng, and Y. Deng, "Holmes: Multimodal agentic diagnosis for mixed-language mobile crashes at industrial scale," arXiv preprint arXiv:2606.21963 (accepted at FSE 2026 Industry Track), 2026. [Online]. Available: https://arxiv.org/abs/2606.21963

[31] R. Wu, H. Zhang, S.-C. Cheung, and S. Kim, "CrashLocator: Locating crashing faults based on crash stacks," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 204–214.

[32] M. Günther, J. Ong, I. Mohr, A. Abdessalem, T. Abel, M. K. Akram, S. Guzman, G. Mastrapas, S. Sturua, B. Wang, M. Werk, N. Wang, and H. Xiao, "Jina embeddings 2: 8192-token general-purpose text embeddings for long documents," arXiv preprint arXiv:2310.19923, 2023. [Online]. Available: https://arxiv.org/abs/2310.19923

[33] Z. Jiang, X. Ren, M. Yan, W. Jiang, Y. Li, and Z. Liu, "Issue localization via LLM-driven iterative code graph searching," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 3034–3045.

[34] Z. Yu, H. Zhang, Y. Zhao, H. Huang, M. Yao, K. Ding, and J. Zhao, "OrcaLoca: An LLM agent framework for software issue localization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025, pp. 73 416–73 436.

[35] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604.

[36] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the International Conference*

on Learning Representations (ICLR), 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[37] J. Zhou, H. Zhang, and D. Lo, "Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports," in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 14–24.

[38] R. K. Saha, M. Lease, S. Khurshid, and D. E. Perry, "Improving bug localization using structured information retrieval," in *Proceedings of the 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2013, pp. 345–355.

[39] M. Wen, R. Wu, and S.-C. Cheung, "Locus: Locating bugs from software changes," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 262–273.

[40] C.-P. Wong, Y. Xiong, H. Zhang, D. Hao, L. Zhang, and H. Mei, "Boosting bug-report-oriented fault localization with segmentation and stack-trace analysis," in *Proceedings of the 2014 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2014, pp. 181–190.

[41] Z. Jiang, J. Liu, Y. Li, H. Huang, X. He, T. Zhang, J. Chen, Y. Li, R. Shi, and M. R. Lyu, "LogPilot: Intent-aware and scalable alert diagnosis for large-scale online service systems," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 3299–3310.

[42] S. Shan, Y. Huo, Y. Su, Y. Li, D. Li, and Z. Zheng, "Face it yourselves: An LLM-based two-stage strategy to localize configuration errors via logs," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 13–25.

[43] D. Roy, X. Zhang, R. Bhave, C. Bansal, P. Las-Casas, R. Fonseca, and S. Rajmohan, "Exploring LLM-based agents for root cause analysis," arXiv preprint arXiv:2403.04123, 2024. [Online]. Available: https://arxiv.org/abs/2403.04123

[44] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "OpenRCA: Can large language models locate the root cause of software failures?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=M4qNIzQYpd

[45] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proceedings of the ACM Web Conference (WWW Companion)*, 2025, pp. 777–780.